

Percursos em Árvores Binárias

Estrutura de Dados Avançada — QXD0015



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Prof. Atílio Gomes Luiz
gomes.atilio@ufc.br

Universidade Federal do Ceará

1º semestre/2025



Percursos em Árvores Binárias

- Muitas operações em árvores binárias envolvem o percurso de todas as subárvores, com execução de alguma ação de tratamento em cada nó.

Percursos em Árvores Binárias

- Muitas operações em árvores binárias envolvem o percurso de todas as subárvores, com execução de alguma ação de tratamento em cada nó.
- É comum percorrer uma árvore em uma das seguintes ordens:
 - **pré-ordem:**
 - **visita raiz**, percorre `r->left`, percorre `r->right`

Percursos em Árvores Binárias

- Muitas operações em árvores binárias envolvem o percurso de todas as subárvores, com execução de alguma ação de tratamento em cada nó.
- É comum percorrer uma árvore em uma das seguintes ordens:
 - **pré-ordem:**
 - **visita raiz**, percorre `r->left`, percorre `r->right`
 - **ordem simétrica:**
 - percorre `r->left`, **visita raiz**, percorre `r->right`

Percursos em Árvores Binárias

- Muitas operações em árvores binárias envolvem o percurso de todas as subárvores, com execução de alguma ação de tratamento em cada nó.
- É comum percorrer uma árvore em uma das seguintes ordens:
 - **pré-ordem:**
 - **visita raiz**, percorre `r->left`, percorre `r->right`
 - **ordem simétrica:**
 - percorre `r->left`, **visita raiz**, percorre `r->right`
 - **pós-ordem:**
 - percorre `r->left`, percorre `r->right`, **visita raiz**

Percorrendo os nós — Pré-ordem

A pré-ordem

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda

Percorrendo os nós — Pré-ordem

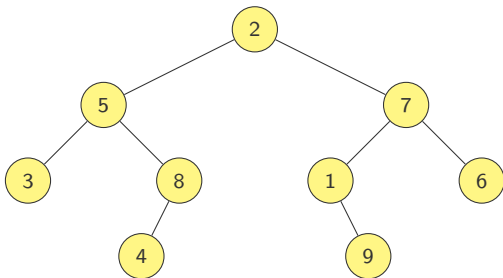
A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

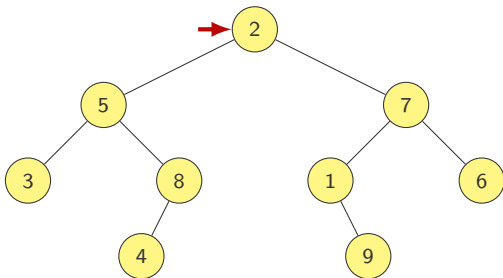


Ex:

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

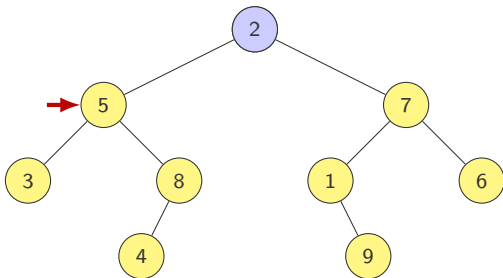


Ex:

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

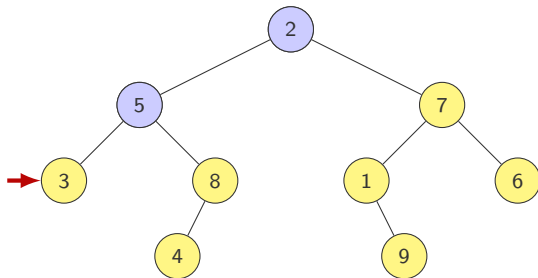


Ex: 2,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

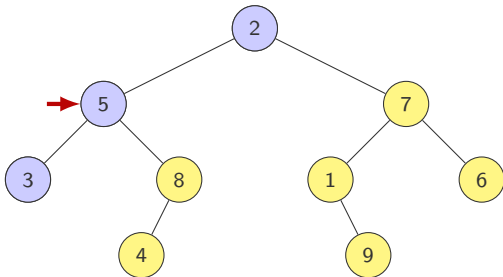


Ex: 2, 5,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

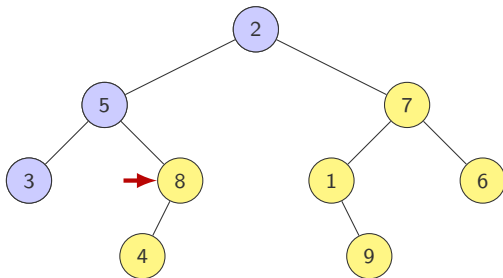


Ex: 2, 5, 3,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

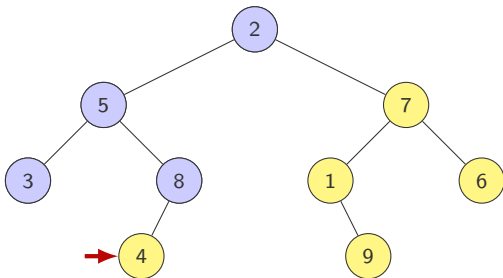


Ex: 2, 5, 3,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

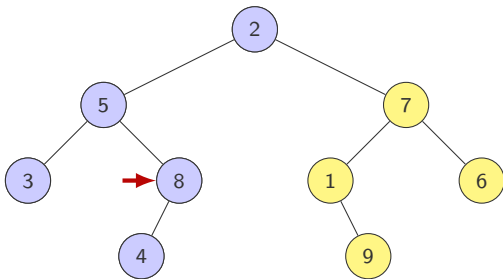


Ex: 2, 5, 3, 8,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

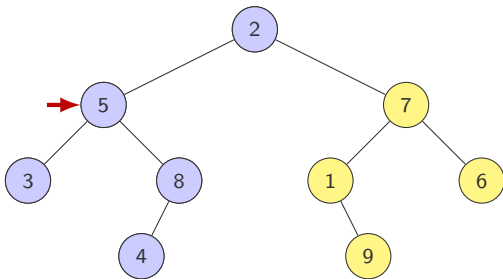


Ex: 2, 5, 3, 8, 4,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

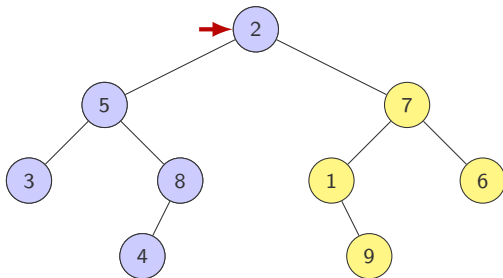


Ex: 2, 5, 3, 8, 4,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

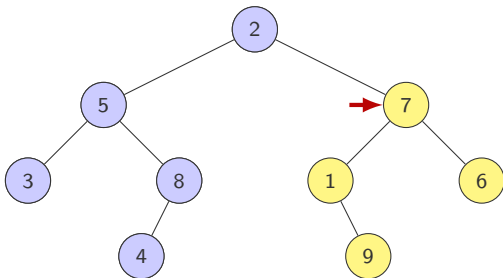


Ex: 2, 5, 3, 8, 4,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

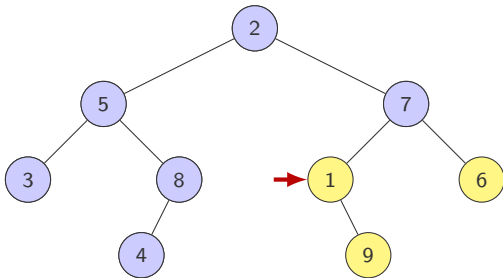


Ex: 2, 5, 3, 8, 4,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

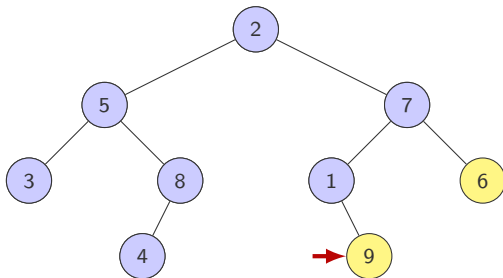


Ex: 2, 5, 3, 8, 4, 7,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

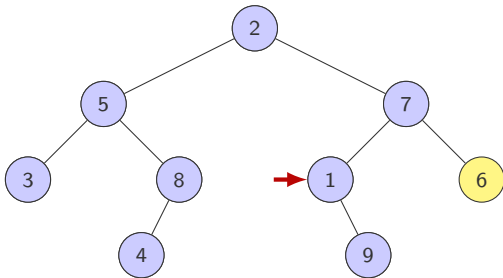


Ex: 2, 5, 3, 8, 4, 7, 1,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

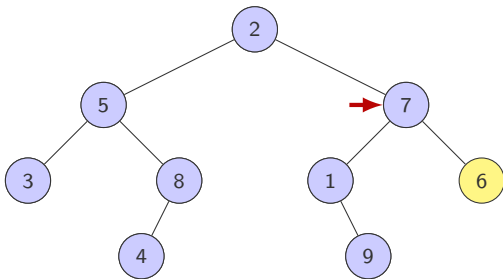


Ex: 2, 5, 3, 8, 4, 7, 1, 9,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

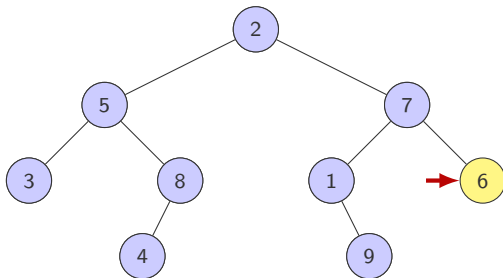


Ex: 2, 5, 3, 8, 4, 7, 1, 9,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

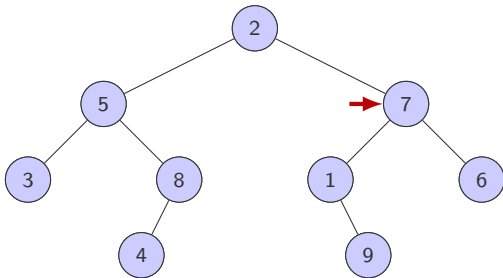


Ex: 2, 5, 3, 8, 4, 7, 1, 9,

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita

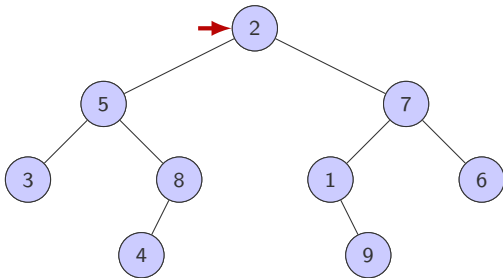


Ex: 2, 5, 3, 8, 4, 7, 1, 9, 6

Percorrendo os nós — Pré-ordem

A pré-ordem

- primeiro visita (processa) a raiz
- depois a subárvore esquerda
- depois a subárvore direita



Ex: 2, 5, 3, 8, 4, 7, 1, 9, 6

Pré-ordem

Algorithm preorder(ptr)

Require: ptr (pointer to node)

- 1: **if** ptr $\neq$ NULL **then**
 - 2: visit(ptr)
 - 3: preorder(ptr→left)
 - 4: preorder(ptr→right)
 - 5: **end if**
-

Percorrendo os nós — Pós-ordem

A pós-ordem

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita

Percorrendo os nós — Pós-ordem

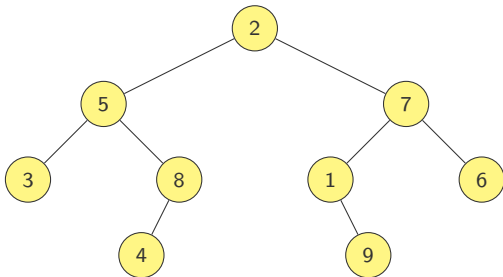
A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

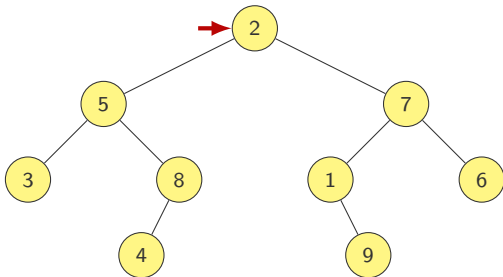


Ex:

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

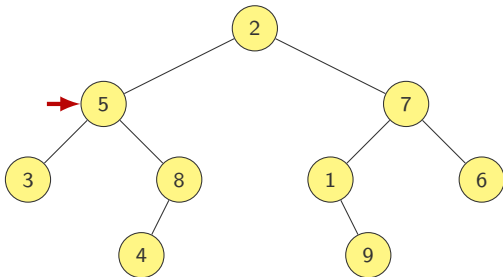


Ex:

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

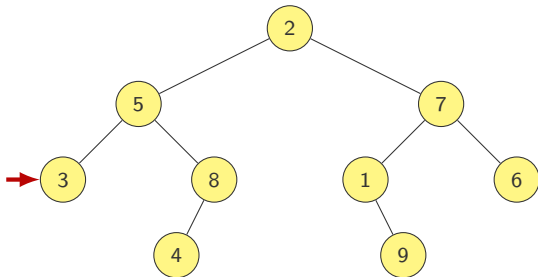


Ex:

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

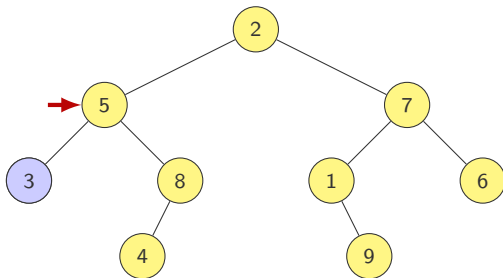


Ex:

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

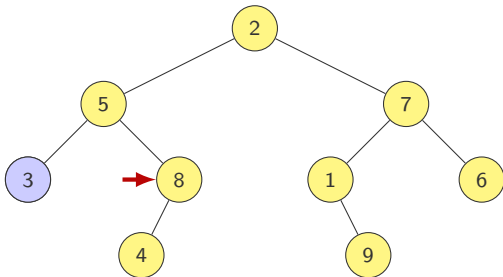


Ex: 3,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

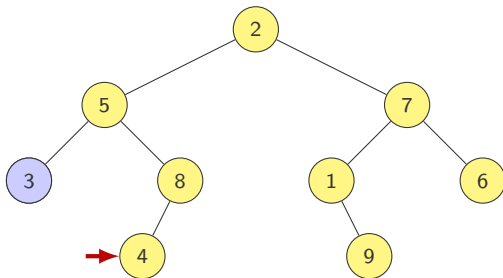


Ex: 3,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

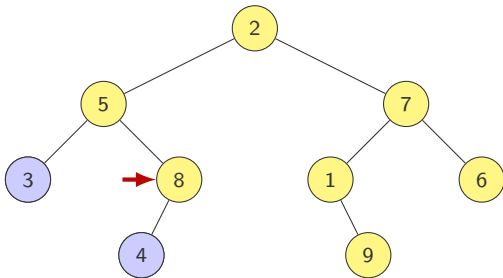


Ex: 3,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

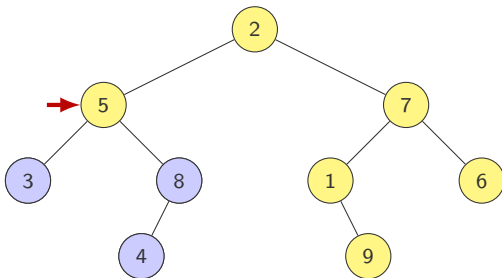


Ex: 3, 4,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

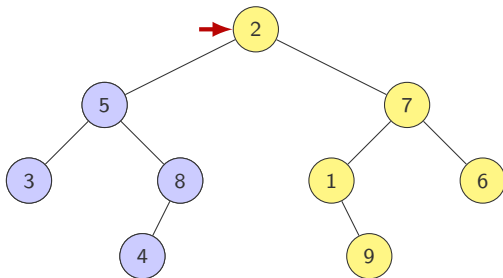


Ex: 3, 4, 8,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

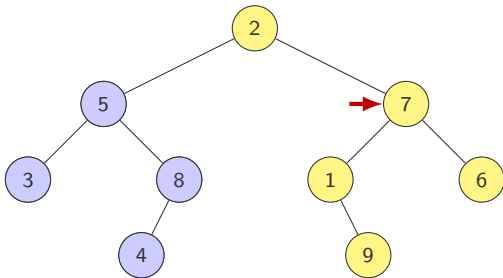


Ex: 3, 4, 8, 5,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

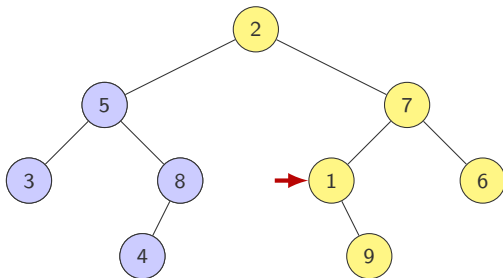


Ex: 3, 4, 8, 5,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

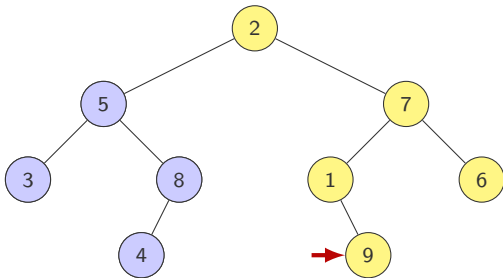


Ex: 3, 4, 8, 5,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

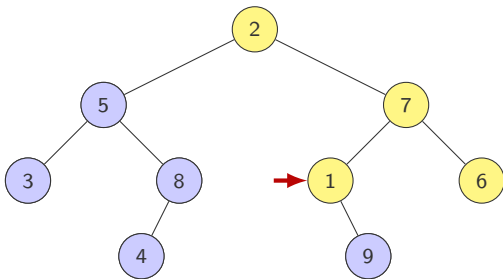


Ex: 3, 4, 8, 5,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

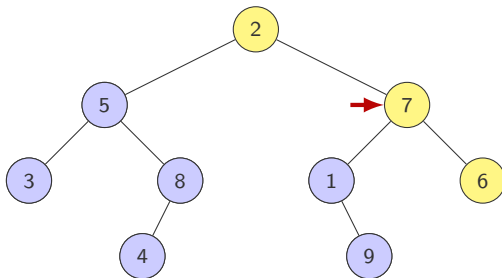


Ex: 3, 4, 8, 5, 9,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

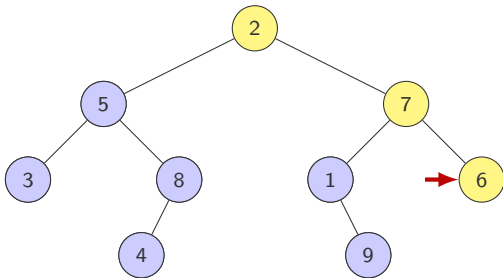


Ex: 3, 4, 8, 5, 9, 1,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

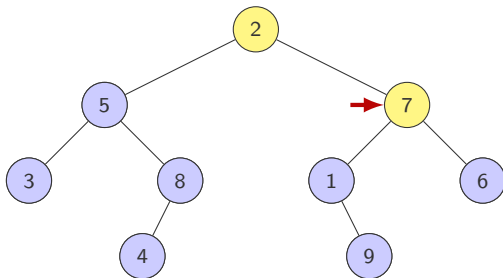


Ex: 3, 4, 8, 5, 9, 1,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

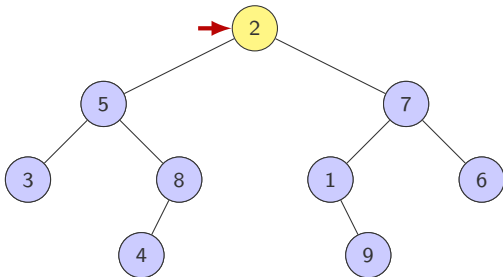


Ex: 3, 4, 8, 5, 9, 1, 6,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz

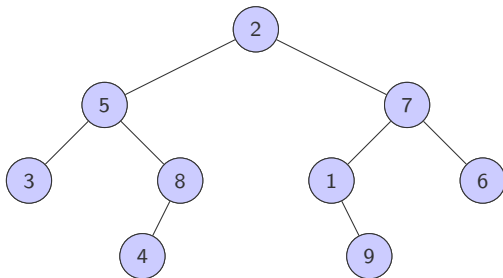


Ex: 3, 4, 8, 5, 9, 1, 6, 7,

Percorrendo os nós — Pós-ordem

A pós-ordem

- primeiro visita a subárvore esquerda
- depois a subárvore direita
- e por último visita a raiz



Ex: 3, 4, 8, 5, 9, 1, 6, 7, 2

Pós-ordem

Algorithm posorder(ptr)

Require: ptr (pointer to node)

- 1: **if** ptr $\neq$ NULL **then**
 - 2: posorder(ptr→left)
 - 3: posorder(ptr→right)
 - 4: visit(ptr)
 - 5: **end if**
-

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz

Percorrendo os nós — Ordem Simétrica

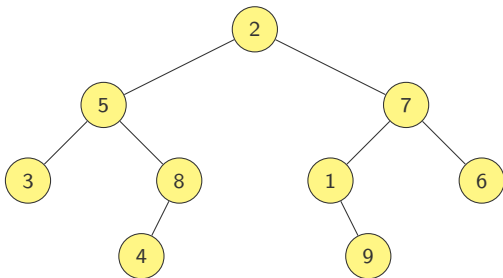
A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

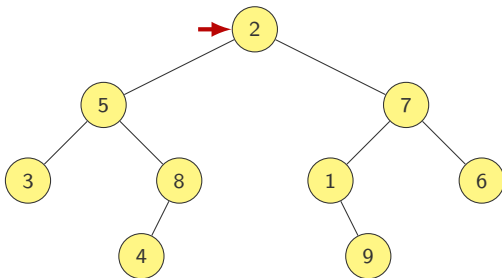


Ex:

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

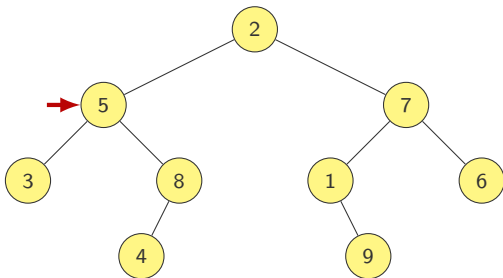


Ex:

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

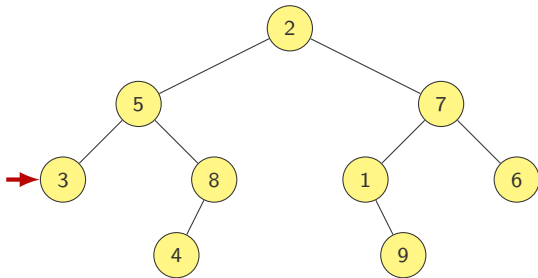


Ex:

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

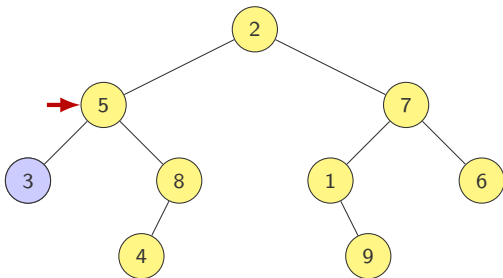


Ex:

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

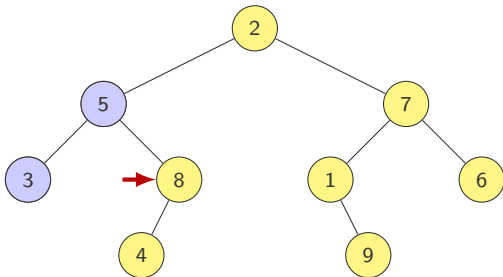


Ex: 3,

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

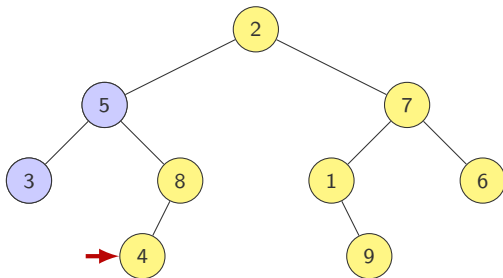


Ex: 3, 5,

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

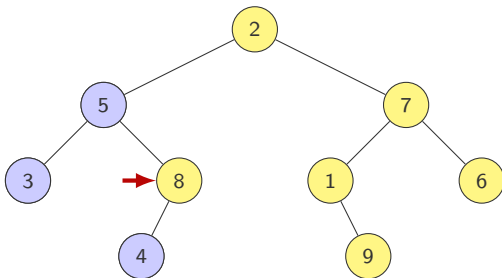


Ex: 3, 5,

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

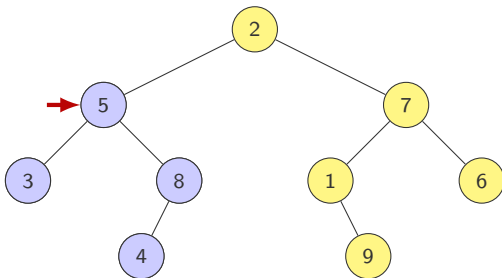


Ex: 3, 5, 4,

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

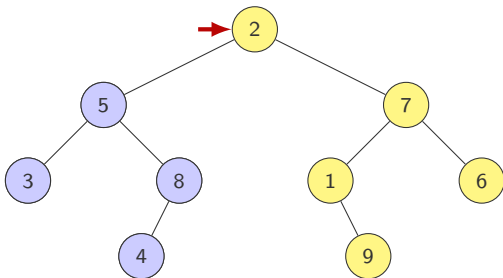


Ex: 3, 5, 4, 8,

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

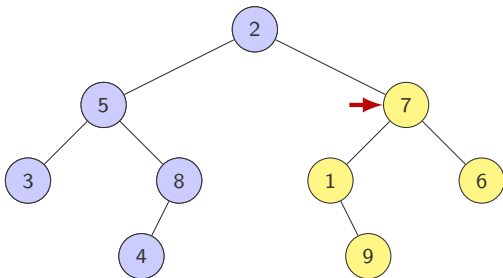


Ex: 3, 5, 4, 8,

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

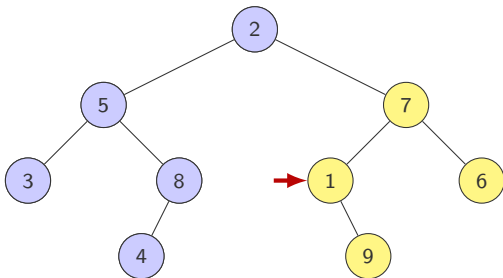


Ex: 3, 5, 4, 8, 2,

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

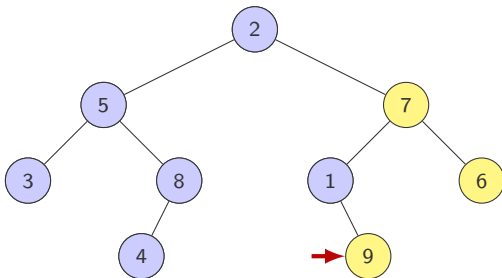


Ex: 3, 5, 4, 8, 2,

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

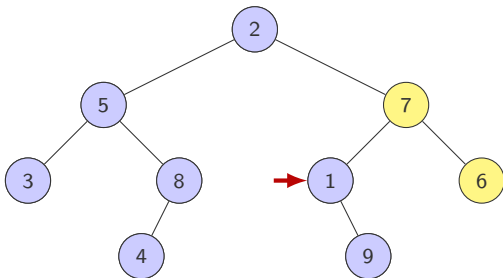


Ex: 3, 5, 4, 8, 2, 1,

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

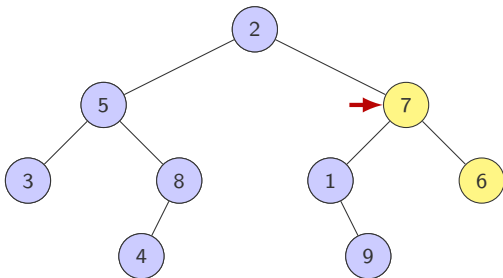


Ex: 3, 5, 4, 8, 2, 1, 9,

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

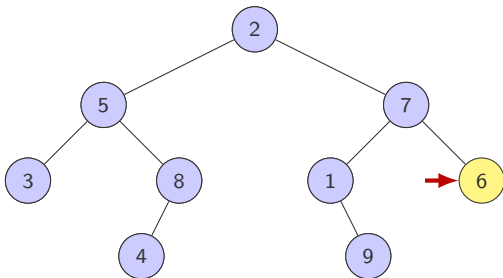


Ex: 3, 5, 4, 8, 2, 1, 9,

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

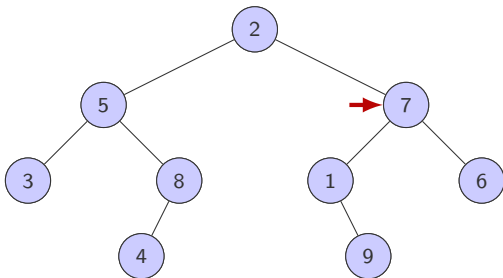


Ex: 3, 5, 4, 8, 2, 1, 9, 7,

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita

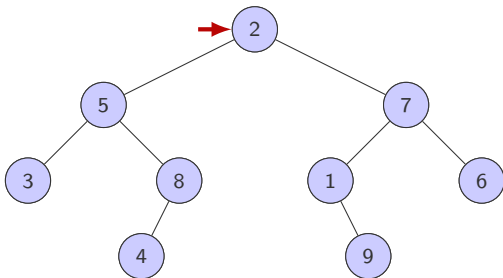


Ex: 3, 5, 4, 8, 2, 1, 9, 7, 6

Percorrendo os nós — Ordem Simétrica

A ordem simétrica

- primeiro visita a subárvore esquerda
- depois visita a raiz
- e por última visita a subárvore direita



Ex: 3, 5, 4, 8, 2, 1, 9, 7, 6

Ordem Simétrica (inorder)

Algorithm inorder(ptr)

Require: ptr (pointer to node)

- 1: **if** ptr $\neq$ NULL **then**
 - 2: inorder(ptr→left)
 - 3: visit(ptr)
 - 4: inorder(ptr→right)
 - 5: **end if**
-

Percurso em largura



Percorrendo os nós (em largura)

O percurso em largura

Percorrendo os nós (em largura)

O percurso em largura

- visita os nós por níveis

Percorrendo os nós (em largura)

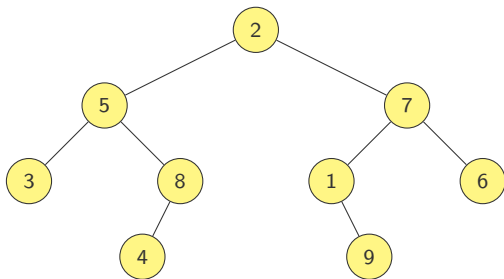
O percurso em largura

- visita os nós por níveis
- da esquerda para a direita

Percorrendo os nós (em largura)

O percurso em largura

- visita os nós por níveis
- da esquerda para a direita

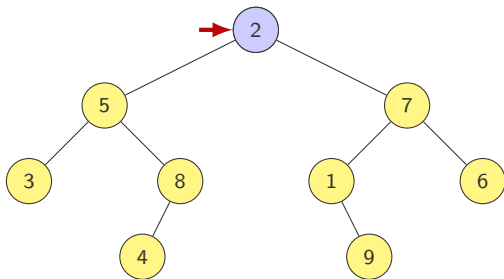


Ex:

Percorrendo os nós (em largura)

O percurso em largura

- visita os nós por níveis
- da esquerda para a direita

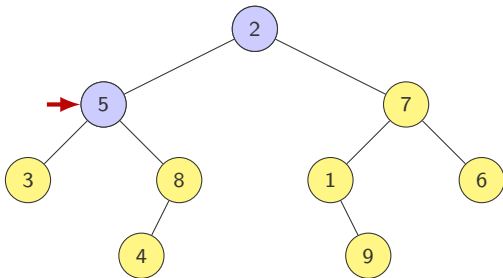


Ex: 2,

Percorrendo os nós (em largura)

O percurso em largura

- visita os nós por níveis
- da esquerda para a direita

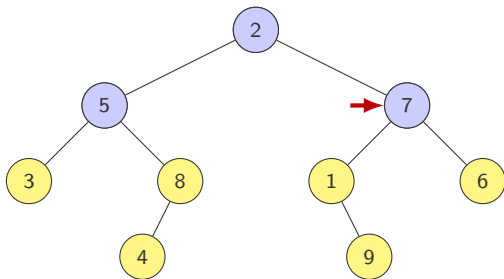


Ex: 2, 5,

Percorrendo os nós (em largura)

O percurso em largura

- visita os nós por níveis
- da esquerda para a direita

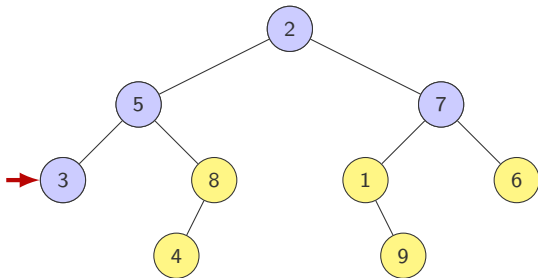


Ex: 2, 5, 7,

Percorrendo os nós (em largura)

O percurso em largura

- visita os nós por níveis
- da esquerda para a direita

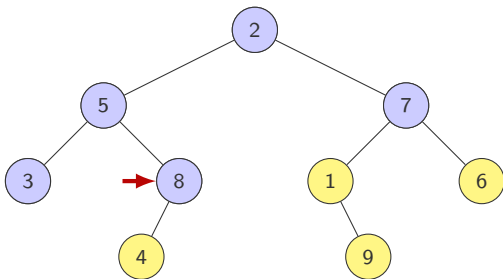


Ex: 2, 5, 7, 3,

Percorrendo os nós (em largura)

O percurso em largura

- visita os nós por níveis
- da esquerda para a direita

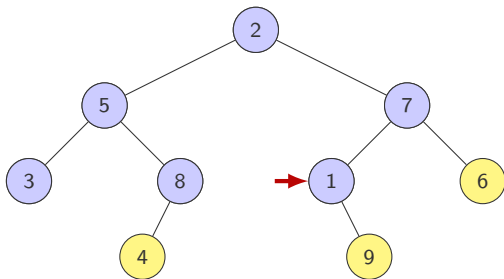


Ex: 2, 5, 7, 3, 8,

Percorrendo os nós (em largura)

O percurso em largura

- visita os nós por níveis
- da esquerda para a direita

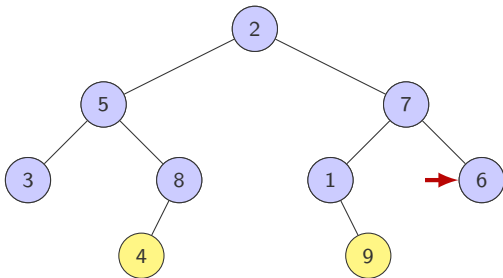


Ex: 2, 5, 7, 3, 8, 1,

Percorrendo os nós (em largura)

O percurso em largura

- visita os nós por níveis
- da esquerda para a direita

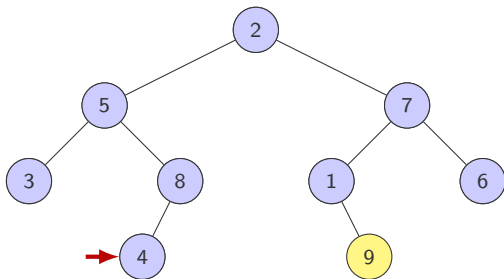


Ex: 2, 5, 7, 3, 8, 1, 6,

Percorrendo os nós (em largura)

O percurso em largura

- visita os nós por níveis
- da esquerda para a direita

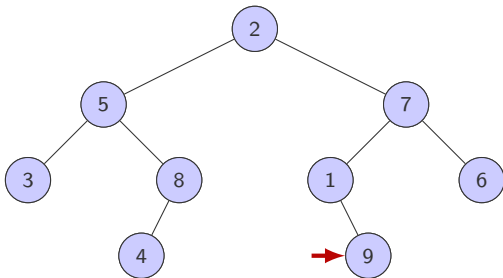


Ex: 2, 5, 7, 3, 8, 1, 6, 4,

Percorrendo os nós (em largura)

O percurso em largura

- visita os nós por níveis
- da esquerda para a direita



Ex: 2, 5, 7, 3, 8, 1, 6, 4, 9

Implementação do percurso em largura

Como implementar a busca em largura?

Implementação do percurso em largura

Como implementar a busca em largura?

- Usamos uma fila

Implementação do percurso em largura

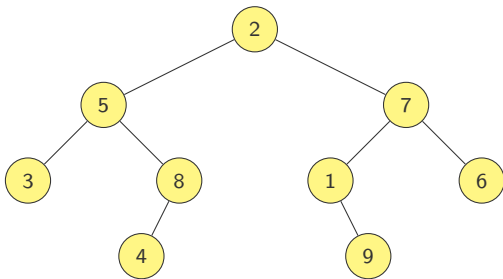
Como implementar a busca em largura?

- Usamos uma fila
- Colocamos a raiz na fila e depois

Implementação do percurso em largura

Como implementar a busca em largura?

- Usamos uma fila
- Colocamos a raiz na fila e depois
- pegamos um elemento da fila e enfileiramos seus filhos

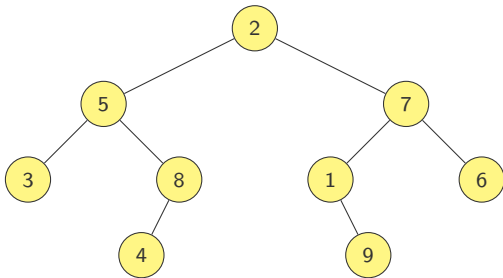


Fila

Implementação do percurso em largura

Como implementar a busca em largura?

- Usamos uma fila
- Colocamos a raiz na fila e depois
- pegamos um elemento da fila e enfileiramos seus filhos

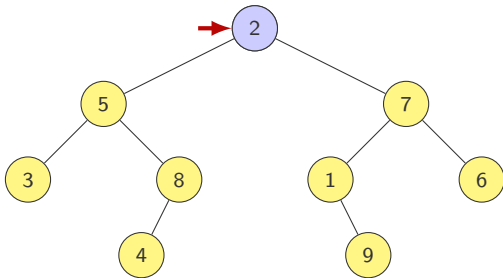


Fila 2

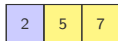
Implementação do percurso em largura

Como implementar a busca em largura?

- Usamos uma fila
- Colocamos a raiz na fila e depois
- pegamos um elemento da fila e enfileiramos seus filhos



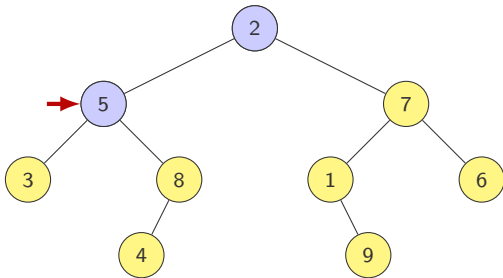
Fila



Implementação do percurso em largura

Como implementar a busca em largura?

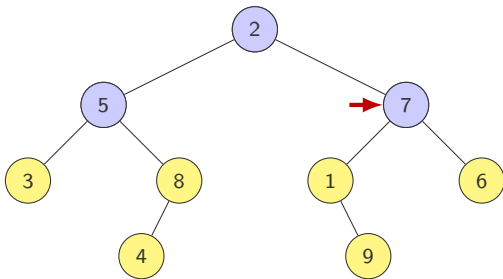
- Usamos uma fila
- Colocamos a raiz na fila e depois
- pegamos um elemento da fila e enfileiramos seus filhos



Implementação do percurso em largura

Como implementar a busca em largura?

- Usamos uma fila
- Colocamos a raiz na fila e depois
- pegamos um elemento da fila e enfileiramos seus filhos



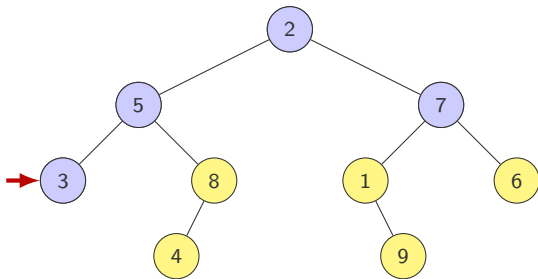
Fila

2	5	7	3	8	1	6
---	---	---	---	---	---	---

Implementação do percurso em largura

Como implementar a busca em largura?

- Usamos uma fila
- Colocamos a raiz na fila e depois
- pegamos um elemento da fila e enfileiramos seus filhos



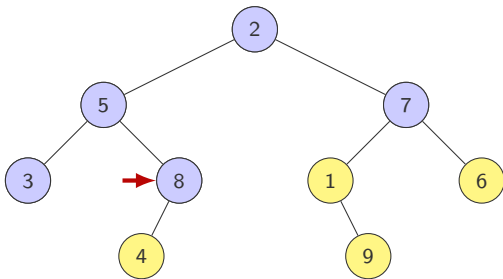
Fila

2	5	7	3	8	1	6
---	---	---	---	---	---	---

Implementação do percurso em largura

Como implementar a busca em largura?

- Usamos uma fila
- Colocamos a raiz na fila e depois
- pegamos um elemento da fila e enfileiramos seus filhos



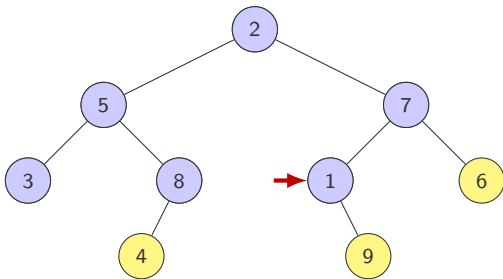
Fila



Implementação do percurso em largura

Como implementar a busca em largura?

- Usamos uma fila
- Colocamos a raiz na fila e depois
- pegamos um elemento da fila e enfileiramos seus filhos



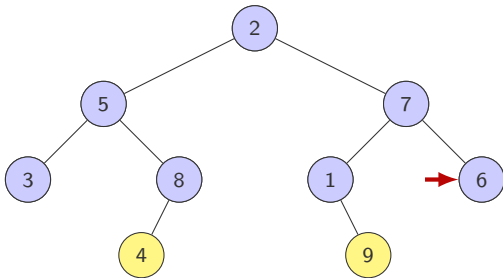
Fila

2	5	7	3	8	1	6	4	9
---	---	---	---	---	---	---	---	---

Implementação do percurso em largura

Como implementar a busca em largura?

- Usamos uma fila
- Colocamos a raiz na fila e depois
- pegamos um elemento da fila e enfileiramos seus filhos



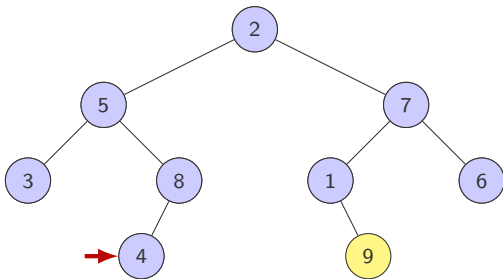
Fila

2	5	7	3	8	1	6	4	9
---	---	---	---	---	---	---	---	---

Implementação do percurso em largura

Como implementar a busca em largura?

- Usamos uma fila
- Colocamos a raiz na fila e depois
- pegamos um elemento da fila e enfileiramos seus filhos



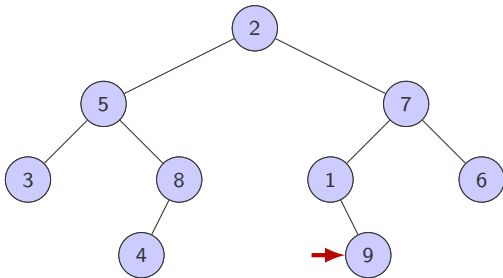
Fila

2	5	7	3	8	1	6	4	9
---	---	---	---	---	---	---	---	---

Implementação do percurso em largura

Como implementar a busca em largura?

- Usamos uma fila
- Colocamos a raiz na fila e depois
- pegamos um elemento da fila e enfileiramos seus filhos



Fila

2	5	7	3	8	1	6	4	9
---	---	---	---	---	---	---	---	---

Percurso em largura

Algorithm levelTraversal()

Require: root (ponteiro para a raiz)

- 1: Cria uma fila vazia Q de ponteiros para nós
 - 2: Q.push(root)
 - 3: **while** Q $\neq \emptyset$ **do**
 - 4: node = Q.front()
 - 5: Q.pop()
 - 6: **if** node $\neq$ NULL **then**
 - 7: visit(node)
 - 8: Q.push(node→left)
 - 9: Q.push(node→right)
 - 10: **end if**
 - 11: **end while**
-

Percursos Iterativos em Árvore



Percurso em pré-ordem — Recursivo

Vimos que o percurso em pré-ordem recursivo é implementado pela seguinte função:

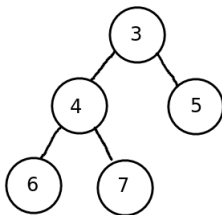
Algorithm preorder(ptr)

Require: ptr (pointer to node)

- 1: **if** ptr $\neq$ NULL **then**
 - 2: visit(ptr)
 - 3: preorder(ptr→left)
 - 4: preorder(ptr→right)
 - 5: **end if**
-

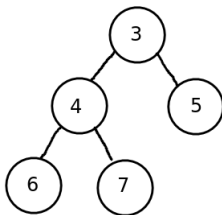
Percurso em pré-ordem — Iterativo

Como implementar a pré-ordem sem usar recursão?



Percurso em pré-ordem — Iterativo

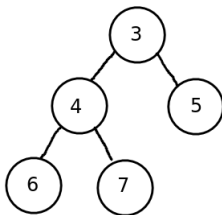
Como implementar a pré-ordem sem usar recursão?



- Na pré-ordem, ao chegarmos ao nó pela primeira vez, nós o visitamos, depois percorremos a esquerda e, só depois a direita.

Percurso em pré-ordem — Iterativo

Como implementar a pré-ordem sem usar recursão?



- Na pré-ordem, ao chegarmos ao nó pela primeira vez, nós o visitamos, depois percorremos a esquerda e, só depois a direita.
- Ao terminar de percorrer a subárvore esquerda do nó, é preciso lembrarmo-nos de que precisamos percorrer a subárvore direita dele.

Percurso em pré-ordem — Iterativo

Algorithm Pré-ordem Iterativo

Require: ponteiro para a raiz da árvore: root

```
1: Cria uma pilha vazia P de ponteiros para nós
2: if root  $\neq$  NULL then
3:   P.push(root)
4: end if
5: while P  $\neq$   $\emptyset$  do
6:   node = P.top()
7:   P.pop()
8:   visit(node)
9:   if node→right  $\neq$  NULL then
10:    P.push(node→right)
11:  end if
12:  if node→left  $\neq$  NULL then
13:    P.push(node→left)
14:  end if
15: end while
```

Percurso em pré-ordem — Iterativo

Algorithm Pré-ordem Iterativo

Require: ponteiro para a raiz da árvore: root

```
1: Cria uma pilha vazia P de ponteiros para nós
2: if root  $\neq$  NULL then
3:   P.push(root)
4: end if
5: while P  $\neq$   $\emptyset$  do
6:   node = P.top()
7:   P.pop()
8:   visit(node)
9:   if node→right  $\neq$  NULL then
10:    P.push(node→right)
11:  end if
12:  if node→left  $\neq$  NULL then
13:    P.push(node→left)
14:  end if
15: end while
```

Por que empilhamos node→right primeiro? E se fosse o contrário?

Percurso em ordem simétrica — Recursivo

O percurso em ordem simétrica (inordem) recursivo é implementado pela seguinte função:

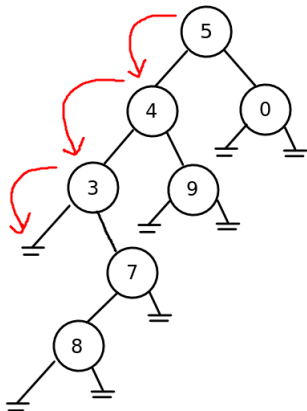
Algorithm inorder(ptr)

Require: ptr (pointer to node)

- 1: **if** ptr $\neq$ NULL **then**
 - 2: inorder(ptr→left)
 - 3: visit(ptr)
 - 4: inorder(ptr→right)
 - 5: **end if**
-

Percurso em ordem simétrica — Dificuldade

Como percorrer em ordem simétrica sem usar recursão?



Percurso em ordem simétrica — Iterativo

Percurso em ordem simétrica — Iterativo

Algorithm ordem simétrica iterativo

Require: ponteiro para a raiz: root

```
1: Cria uma pilha vazia P de ponteiros para nós
2: node = root
3: while P  $\neq \emptyset$  or node  $\neq$  NULL do
4:   if node  $\neq$  NULL then
5:     P.push(node)
6:     node = node→left
7:   else
8:     node = P.top()
9:     P.pop()
10:    visit(node)
11:    node = node→right
12:   end if
13: end while
```

Relembrando: Pós-ordem recursivo

Algorithm post-order(ptr)

Require: ptr (pointer to node)

- 1: **if** ptr $\neq$ NULL **then**
 - 2: post-order(ptr→left)
 - 3: post-order(ptr→right)
 - 4: visit(ptr)
 - 5: **end if**
-

Relembrando: Pós-ordem recursivo

Algorithm post-order(ptr)

Require: ptr (pointer to node)

- 1: **if** ptr $\neq$ NULL **then**
 - 2: post-order(ptr→left)
 - 3: post-order(ptr→right)
 - 4: visit(ptr)
 - 5: **end if**
-

Dos três percursos, a pós-ordem é o mais difícil de tornar iterativo.

Vou usar uma abordagem utilizando um percurso auxiliar, chamado **pré-ordem reversa**.

Pré-ordem reversa

A pré-ordem reversa

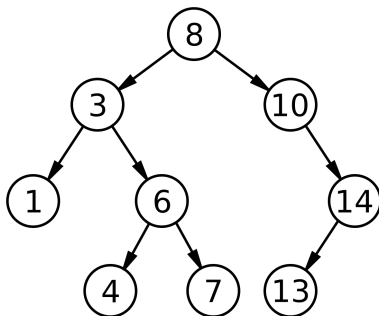
- primeiro visita (processa) a raiz
- depois a subárvore direita
- depois a subárvore esquerda

Pré-ordem reversa

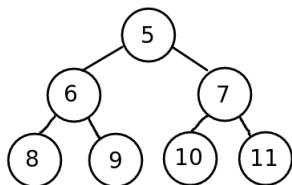
A pré-ordem reversa

- primeiro visita (processa) a raiz
- depois a subárvore direita
- depois a subárvore esquerda

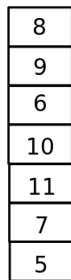
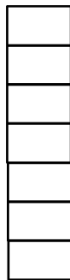
Execute a pré-ordem reversa na árvore abaixo:



Percurso em pós-ordem iterativo



pos-ordem
→
8 9 6 10 11 7 5
←
pré-ordem reversa



Percurso em pós-ordem — Iterativo

Algorithm Pos-Ordem-Iterativo

```
1: Cria pilhas vazias P1 e P2 de ponteiros para nós
2: if root != NULL then P1.push(root)
3: while P1  $\neq \emptyset$  do
4:   node = P1.top()
5:   P1.pop()
6:   P2.push(node)
7:   if node→left != NULL then P1.push(node→left)
8:   if node→right != NULL then P1.push(node→right)
9: end while
10: while P2  $\neq \emptyset$  do
11:   node = P2.top()
12:   P2.pop()
13:   visit(node)
14: end while
```

Exercícios



Exercício

- Escreva uma função recursiva que recebe como entrada o ponteiro para a raiz de uma árvore e retorna uma cópia da árvore. Um cópia não compartilha memória, o que significa que você terá que alocar novos nós e retornar uma nova árvore idêntica à árvore dada como entrada.

O protótipo da função é dado abaixo:

```
Node* copy(Node* node);
```

Exercícios

- Escreva uma função recursiva que recebe como entrada um ponteiro para a raiz de uma árvore binária T_1 e um ponteiro para a raiz de uma árvore binária T_2 e retorna **true** se e somente se as duas árvores binárias forem **idênticas**. Duas árvores são idênticas se os nós estão localizados nas mesmas posições e as chaves também são iguais nas respectivas posições.

O protótipo da função é dado abaixo:

```
bool equal(Node* ptr1, Node* ptr2);
```

Exercícios

- Completar a implementação da árvore binária de busca com as funções:
 - Construtor de cópia
 - operador de atribuição
 - operador de igualdade
 - operador de diferença
 - funções de mínimo e máximo
 - implementação dos variados percursos
 - antecessor
 - sucessor

Exercícios

- Escreva uma função que calcula o número de nós de uma árvore. A função deve obedecer o seguinte protótipo:

```
int bt_size(Node* node);
```

Exercícios

- Escreva uma função que calcula o número de nós de uma árvore. A função deve obedecer o seguinte protótipo:
`int bt_size(Node* node);`
- Escreva uma função que calcula a altura de uma árvore. A função deve obedecer o seguinte protótipo:
`int bt_height(Node* node);`

Exercícios

- Escreva uma função que calcula o número de nós de uma árvore. A função deve obedecer o seguinte protótipo:
`int bt_size(Node* node);`
- Escreva uma função que calcula a altura de uma árvore. A função deve obedecer o seguinte protótipo:
`int bt_height(Node* node);`
- Adicione o campo `height` ao struct `Node`. O campo `height` deve ser do tipo `int`. Implemente a função `bt_height(Node* node)` de modo que ela preencha o campo `height` de cada nó com a altura do nó.

Exercícios

- Um caminho que vai da raiz de uma árvore até um nó qualquer pode ser representado por uma sequência de 0s e 1s, do seguinte modo:
 - toda vez que o caminho “desce para a esquerda” temos um 0; toda vez que “desce para a direita” temos um 1.
 - Diremos que essa sequência de 0s e 1s é o **código** do nó.
- Suponha agora que todo nó de nossa árvore tem um campo adicional `code`, do tipo `std::string`, capaz de armazenar uma cadeia de caracteres de tamanho variável. Escreva uma função que preencha o campo `code` de cada nó com o código do nó.

FIM

